Abrhil

Documentación Técnica

Sistema de Gestión de Préstamos de Libros

Esther Elizabeth Salas Chacón
21-04-2026

Contenido

I. Introducción 2

II. Alcance general del sistema 3

III. Requerimientos funcionales 4

IV. Requerimientos no funcionales 5

V. Arquitectura 6

VI. Reglas de negocio 7

VII. Endpoints 7

VIII. Matriz de pruebas 9

IX. Actores 11

X. Tablas de Casos de uso 12

XI. Diagramas de alto nivel 16

- Diagrama Entidad-Relación 16
- Diagrama de Clases 16
- Flujo general del préstamo 17
- Flujo de Devolución 18

XII. Criterios de aceptación 19

XIII. Pruebas (Postman) 19

XIV. Levantamiento de Proyecto 20

1. Clonar el repositorio 20
2. Configurar variables de entorno 20
3. Construir y levantar los contenedores 20
4. Ejecutar migraciones 20
5. Crear superusuario 20
6. Acceder al sistema 20

XV. Conclusión 21

I. Introducción

Este documento presenta la solución desarrollada para la prueba técnica correspondiente al Sistema de Gestión de Préstamos de Libros, una API REST construida utilizando Django REST Framework, PostgreSQL y Docker. El objetivo principal de esta implementación es ofrecer una solución robusta, escalable y alineada con buenas prácticas de desarrollo backend, que permita gestionar de manera eficiente el catálogo de libros y el ciclo de vida de los préstamos.

La aplicación contempla no solo las operaciones básicas de creación, consulta, actualización y eliminación de recursos, sino también la implementación de lógica de negocio más avanzada, como el control de disponibilidad de libros, la gestión de devoluciones parciales, la extensión de préstamos y la validación de reglas que garantizan la integridad de la información. Adicionalmente, se incorporaron mecanismos de control de acceso basados en roles, asegurando que cada usuario únicamente pueda interactuar con los recursos que le corresponden.

Como parte del proceso de desarrollo, se incluyó una estrategia de pruebas funcionales mediante Postman, abarcando escenarios exitosos, casos de error y validaciones de seguridad, lo que permite verificar el correcto funcionamiento del sistema desde una perspectiva integral. Esta documentación describe el alcance, los requerimientos, la arquitectura, las reglas de negocio y las pruebas realizadas, proporcionando una visión completa de la solución implementada.

II. Alcance general del sistema

El sistema desarrollado abarca la gestión integral de libros y préstamos dentro de un entorno controlado mediante una API REST, permitiendo a los usuarios interactuar con el catálogo de libros y realizar operaciones relacionadas con el préstamo de los mismos. En este contexto, la solución permite administrar el inventario de libros, incluyendo la creación, consulta, actualización y eliminación de registros, así como la asociación opcional de imágenes para cada recurso, lo que facilita su identificación.

Asimismo, el sistema ofrece capacidades de filtrado sobre el catálogo, permitiendo realizar consultas específicas por autor o año de publicación, optimizando la búsqueda de información. En cuanto a la gestión de préstamos, los usuarios autenticados pueden solicitar libros disponibles, generando registros que reflejan el estado del préstamo, su fecha de creación y su evolución a lo largo del tiempo.

El sistema contempla escenarios más avanzados dentro del ciclo de vida del préstamo, como la posibilidad de realizar devoluciones parciales, en las cuales se actualiza dinámicamente la disponibilidad del libro y el estado del préstamo, así como devoluciones completas que finalizan el proceso. De igual forma, se permite la extensión de préstamos activos o parcialmente devueltos, ajustando la fecha de vencimiento conforme a las reglas definidas.

Finalmente, el alcance incluye la implementación de un esquema de control de acceso basado en roles, en el cual los usuarios administradores tienen la capacidad de gestionar el catálogo completo de libros y consultar todos los préstamos, mientras que los usuarios estándar únicamente pueden interactuar con sus propios registros. Este enfoque garantiza la seguridad, integridad y confidencialidad de la información manejada por el sistema.

III. Requerimientos funcionales

Tabla 1 : Requerimientos Funcionales			
ID	**Requerimiento**	**Descripción**	**Prioridad**
RF01	Registrar libros	Permite a los usuarios administradores crear nuevos libros en el sistema con información como título, autor, año, descripción, imagen y control de copias.	Alta
RF02	Consultar libros	Permite visualizar el listado de libros disponibles en el sistema.	Alta
RF03	Filtrar libros por autor	Permite consultar libros utilizando el nombre del autor como criterio de búsqueda.	Media
RF04	Filtrar libros por año	Permite consultar libros utilizando el año de publicación como filtro.	Media
RF05	Actualizar libros	Permite a los administradores modificar parcialmente la información de un libro existente.	Alta
RF06	Eliminar libros	Permite a los administradores eliminar libros del sistema.	Media
RF07	Registrar préstamos	Permite a los usuarios autenticados solicitar préstamos de libros disponibles.	Alta
RF08	Consultar préstamos	Permite a los usuarios consultar sus propios préstamos y a los administradores consultar todos los registros.	Alta
RF09	Devolución parcial	Permite devolver parcialmente las copias prestadas de un libro, actualizando el estado del préstamo.	Alta
RF10	Devolución total	Permite finalizar un préstamo mediante la devolución completa del libro.	Alta
RF11	Extender préstamo	Permite ampliar la duración de un préstamo activo o parcialmente devuelto.	Media
RF12	Restricción de gestión de libros	Solo los usuarios administradores pueden crear, actualizar o eliminar libros.	Alta
RF13	Restricción de acceso a préstamos	Un usuario solo puede acceder y operar sobre sus propios préstamos.	Alta

IV. Requerimientos no funcionales

Tabla 2: Requerimientos No Funcionales			
ID	**Requerimiento**	**Descripción**	**Prioridad**
RNF01	Uso de Django REST Framework	La API debe estar desarrollada utilizando Django REST Framework.	Alta
RNF02	Uso de PostgreSQL	La persistencia de datos debe gestionarse mediante PostgreSQL.	Alta
RNF03	Contenedorización con Docker	La aplicación debe ejecutarse mediante contenedores Docker para facilitar su despliegue.	Alta
RNF04	API basada en JSON	Todas las respuestas y solicitudes deben manejar formato JSON.	Alta
RNF05	Autenticación y autorización	El sistema debe requerir autenticación para operaciones protegidas y aplicar control de acceso por roles.	Alta
RNF06	Pruebas en Postman	Se debe incluir una colección de Postman con escenarios funcionales, de error y seguridad.	Alta

V. Arquitectura

El sistema sigue una arquitectura basada en API REST utilizando Django REST Framework, organizada bajo el patrón MVT (Model-View-Template), adaptado para servicios backend.

Componentes principales:

- Modelos (Models):

Representan la estructura de datos y la interacción con la base de datos PostgreSQL. Aquí se definen entidades como Book y Loan, junto con sus relaciones.

- Serializadores (Serializers):

Encargados de la validación de datos de entrada y transformación de datos entre formatos JSON y objetos del modelo. También contienen validaciones de negocio específicas.

- Vistas (ViewSets):

Gestionan las solicitudes HTTP, delegando la validación a los serializers y la persistencia a los modelos. Implementan la lógica de flujo del sistema.

- Permisos (Permissions):

Controlan el acceso a los recursos mediante roles, diferenciando entre administradores y usuarios autenticados.

- Base de datos:

PostgreSQL como motor relacional, garantizando consistencia y eficiencia en el manejo de datos.

- Contenedores:

Docker permite la ejecución del sistema en entornos aislados, facilitando la portabilidad y despliegue.

- Flujo general de una solicitud:

Cliente → Endpoint (ViewSet) → Serializer (validación) → Modelo (persistencia) → Respuesta JSON

Esta arquitectura permite mantener una separación clara de responsabilidades, facilitando la escalabilidad, mantenibilidad y pruebas del sistema.

VI. Reglas de negocio

El sistema implementa las siguientes reglas de negocio:

- No se pueden registrar préstamos si no existen copias disponibles del libro solicitado.
- La cantidad solicitada en un préstamo debe ser mayor a cero.
- Al crear un préstamo, la cantidad de copias disponibles del libro se reduce según la cantidad solicitada.
- Una devolución parcial incrementa las copias disponibles del libro de manera proporcional.
- Una devolución total actualiza el estado del préstamo a "returned" y restablece completamente las copias correspondientes.
- No se permite devolver más copias de las que fueron prestadas.
- No se permite registrar devoluciones sin indicar la cantidad a devolver.
- No se permite extender un préstamo que ya ha sido devuelto completamente.
- Los préstamos pueden extenderse únicamente si se encuentran en estado activo o parcial.
- Solo el usuario propietario del préstamo puede realizar acciones sobre el mismo.
- Los usuarios administradores pueden visualizar todos los préstamos del sistema.
- Solo los usuarios con rol administrador pueden crear, actualizar o eliminar libros.
- Las imágenes asociadas a los libros deben ser archivos válidos de tipo imagen y no exceder los 2 MB de tamaño.

VII. Endpoints

A continuación, se describen los principales endpoints de la API, los cuales fueron utilizados en las pruebas funcionales mediante la colección de Postman.

Libros:

GET /api/books/

Obtiene el listado de libros disponibles.

POST /api/books/

Permite a un administrador registrar un nuevo libro.

GET /api/books/{id}/

Obtiene el detalle de un libro específico.

PATCH /api/books/{id}/

Permite actualizar parcialmente la información de un libro (solo administrador).

DELETE /api/books/{id}/

Permite eliminar un libro del sistema (solo administrador). Responde con 204 No Content.

GET /api/books/?author={nombre}

Permite filtrar libros por autor.

GET /api/books/?year={año}

Permite filtrar libros por año de publicación.

Préstamos:
GET /api/loans/

Obtiene los préstamos del usuario autenticado. Los administradores pueden ver todos los préstamos.

POST /api/loans/

Permite crear un préstamo de uno o varios libros disponibles.

GET /api/loans/{id}/

Obtiene el detalle de un préstamo específico.

POST /api/loans/{id}/partial_return/

Permite registrar la devolución parcial de un préstamo.

POST /api/loans/{id}/return_book/

Permite registrar la devolución total de un préstamo.

POST /api/loans/{id}/extend/

Permite extender la duración de un préstamo activo o parcial.

VIII. Matriz de pruebas

Tabla 3: Matriz de Prueba RF -CA				
ID	**Requerimiento Funcional**	**Criterio de Aceptación (CA)**	**Caso de Prueba (CP)**	**Resultado Esperado**
RF01	Registrar libros	El sistema permite crear libros con datos válidos por un administrador, incluyendo la carga opcional de una imagen asociada	POST /api/books/ con usuario admin y archivo de imagen válido	201 Created
RF02	Consultar libros	El sistema permite listar todos los libros	GET /api/books/	200 OK
RF03	Filtrar libros por autor	El sistema permite filtrar libros por nombre de autor	GET /api/books/?author=Robert	200 OK con resultados filtrados
RF04	Filtrar libros por año	El sistema permite filtrar libros por año	GET /api/books/?year=2008	200 OK con resultados filtrados
RF05	Actualizar libros	El administrador puede modificar parcialmente un libro	PATCH /api/books/{id}/	200 OK con datos actualizados
RF06	Eliminar libros	El administrador puede eliminar un libro	DELETE /api/books/{id}/	204 No Content
RF07	Registrar préstamos	Un usuario autenticado puede crear un préstamo si hay disponibilidad	POST /api/loans/ con quantity válida	201 Created
RF08	Registrar préstamos sin stock	No se permite crear préstamos si no hay copias disponibles	POST /api/loans/ con libro sin disponibilidad	400 Bad Request
RF09	Consultar préstamos	El usuario solo puede consultar sus propios préstamos	GET /api/loans/{id}/ con otro usuario	404 Not Found
RF10	Devolución parcial	Permite devolver parcialmente un préstamo	POST /api/loans/{id}/partial_return/ con quantity válida	200 OK, status = partial

RF11	Devolución parcial inválida	No permite devolución sin cantidad	POST partial_return con body vacío	400 Bad Request
RF12	Devolución parcial inválida	No permite valores negativos o cero	POST partial_return con quantity <= 0	400 Bad Request
RF13	Devolución parcial inválida	No permite devolver más de lo pendiente	POST partial_return con quantity mayor al pendiente	400 Bad Request
RF14	Devolución total	Permite devolver completamente un préstamo	POST /api/loans/{id}/return_book/	200 OK, status = returned
RF15	Extender préstamo	Permite extender préstamos activos o parciales	POST /api/loans/{id}/extend/	200 OK
RF16	Extender préstamo inválido	No permite extender un préstamo ya devuelto	POST extend sobre préstamo returned	400 Bad Request
RF17	Seguridad en libros	Solo administradores pueden crear libros	POST /api/books/ con usuario normal	403 Forbidden
RF18	Seguridad en préstamos	Un usuario no puede acceder a préstamos ajenos	GET /api/loans/{id}/ con otro usuario	404 Not Found
RF19	Seguridad en préstamos	Un usuario no puede modificar préstamos ajenos	partial_return, return_book o extend con otro usuario	404 Not Found

IX. Actores

Tabla 4: Descripción del actor Administrador/ Privilegios:	
Campo	Descripción
Actor	Administrador
Rol	Responsable de la gestión del catálogo de libros y de la supervisión general de los préstamos registrados en el sistema.
Responsabilidades	• Crear libros. • Actualizar libros. • Eliminar libros. • Consultar el catálogo de libros. • Consultar todos los préstamos registrados. • Validar la información general del sistema.
Precondiciones	Debe contar con credenciales válidas y permisos de tipo administrador o staff.
Postcondiciones	Los cambios realizados sobre los libros se reflejan en el catálogo del sistema y pueden impactar en las operaciones de préstamo.

Tabla 5: Descripción del actor Usuario autenticado/Privilegios:	
Campo	Descripción
Actor	Usuario autenticado
Rol	Responsable de consultar libros y gestionar sus propios préstamos dentro del sistema.
Responsabilidades	• Consultar el catálogo de libros. • Filtrar libros por autor o año. • Solicitar préstamos. • Consultar sus propios préstamos. • Registrar devoluciones parciales. • Registrar devoluciones completas. • Extender préstamos activos o parciales.
Precondiciones	Debe contar con credenciales válidas e iniciar sesión en el sistema.
Postcondiciones	Las acciones realizadas actualizan el estado del préstamo y, en su caso, la disponibilidad del libro en inventario.

Tabla 6: Descripción del actor Usuario no autenticado/Privilegios: Activados	
Campo	Descripción
Actor	Usuario no autenticado
Rol	Actor externo con acceso limitado a la consulta pública del catálogo de libros.
Responsabilidades	• Consultar el catálogo de libros.• Visualizar información general de los libros disponibles.
Precondiciones	No requiere autenticación para acceder a los recursos públicos habilitados.

Postcondiciones	Puede visualizar información de lectura, pero no puede modificar datos ni registrar préstamos.

X. Tablas de Casos de uso

Tabla 7. Especificación del caso de uso CU01 – Gestionar libros	
Campo	Descripción
Caso de uso	CU01 – Gestionar libros
Actores	Administrador
Tipo	Básico
Propósito	Permitir al administrador registrar, actualizar y eliminar libros dentro del sistema.
Precondiciones	1. El administrador debe estar autenticado.2. Debe contar con permisos de administración.
Postcondiciones	1. El catálogo de libros queda actualizado.2. La información registrada queda disponible para consulta y préstamo.
Escenario de éxito	1. El administrador accede al módulo de libros.2. Selecciona la operación deseada: crear, actualizar o eliminar.3. Ingresa o modifica la información correspondiente.4. El sistema valida los datos.5. El sistema registra correctamente la operación.
Extensiones	Validación de imagen inválida, datos incompletos o intento de acceso sin permisos.

Tabla 8. Especificación del caso de uso CU02 – Consultar catálogo de libros	
Campo	Descripción
Caso de uso	CU02 – Consultar catálogo de libros
Actores	Usuario autenticado, Usuario no autenticado, Administrador
Tipo	Básico
Propósito	Permitir la consulta del catálogo general de libros disponibles en el sistema.
Precondiciones	El sistema debe contar con libros registrados.
Postcondiciones	El actor visualiza la información disponible del catálogo de libros.
Escenario de éxito	1. El actor accede al endpoint de consulta de libros.2. El sistema recupera la información del catálogo.3. El sistema muestra el listado de libros disponibles.
Extensiones	Consulta con catálogo vacío.

Tabla 9. Especificación del caso de uso CU03 – Filtrar libros	
Campo	Descripción
Caso de uso	CU03 – Filtrar libros
Actores	Usuario autenticado, Usuario no autenticado, Administrador
Tipo	Básico
Propósito	Permitir filtrar el catálogo de libros mediante criterios de búsqueda como autor o año.
Precondiciones	Deben existir libros registrados en el sistema.
Postcondiciones	El sistema retorna únicamente los libros que cumplen con el criterio solicitado.
Escenario de éxito	1. El actor accede a la consulta de libros.2. Ingresa un filtro por autor o año.3. El sistema procesa el criterio de búsqueda.4. El sistema muestra los resultados filtrados.
Extensiones	No se encuentran coincidencias para el filtro solicitado.

Tabla 10. Especificación del caso de uso CU04 – Registrar préstamo	
Campo	Descripción
Caso de uso	CU04 – Registrar préstamo
Actores	Usuario autenticado
Tipo	Básico
Propósito	Permitir a un usuario autenticado registrar un préstamo de un libro disponible.
Precondiciones	1. El usuario debe estar autenticado.2. El libro debe tener copias disponibles.
Postcondiciones	1. Se crea un registro de préstamo.2. La disponibilidad del libro se reduce según la cantidad solicitada.
Escenario de éxito	1. El usuario selecciona un libro disponible.2. Indica la cantidad a prestar.3. El sistema valida disponibilidad.4. El sistema registra el préstamo.5. El sistema actualiza las copias disponibles.
Extensiones	No hay stock suficiente o la cantidad solicitada es inválida.

Tabla 11. Especificación del caso de uso CU05 – Consultar préstamos	
Campo	Descripción
Caso de uso	CU05 – Consultar préstamos
Actores	Usuario autenticado, Administrador
Tipo	Básico

Propósito	Permitir consultar los préstamos registrados en el sistema según el rol del usuario.
Precondiciones	El actor debe estar autenticado.
Postcondiciones	El sistema muestra los préstamos visibles para el actor según sus permisos.
Escenario de éxito	1. El actor accede al módulo de préstamos.2. El sistema valida el rol del usuario.3. Si es administrador, muestra todos los préstamos.4. Si es usuario normal, muestra únicamente sus préstamos.
Extensiones	El usuario intenta acceder a un préstamo que no le pertenece.

Tabla 12. Especificación del caso de uso CU06 – Registrar devolución parcial	
Campo	Descripción
Caso de uso	CU06 – Registrar devolución parcial
Actores	Usuario autenticado
Tipo	Básico
Propósito	Permitir al usuario devolver parcialmente las copias asociadas a un préstamo.
Precondiciones	1. El usuario debe estar autenticado.2. El préstamo debe encontrarse activo o parcial.3. Debe existir cantidad pendiente por devolver.
Postcondiciones	1. Se incrementa parcialmente la disponibilidad del libro.2. El estado del préstamo se actualiza a parcial o devuelto, según corresponda.
Escenario de éxito	1. El usuario accede al préstamo.2. Indica la cantidad a devolver.3. El sistema valida la cantidad.4. El sistema actualiza el préstamo.5. El sistema incrementa la disponibilidad del libro.
Extensiones	Cantidad vacía, inválida, negativa o mayor a la pendiente.

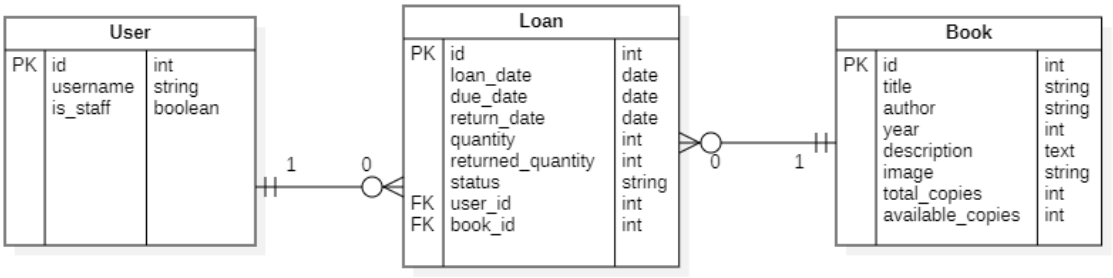
Tabla 13. Especificación del caso de uso CU07 – Registrar devolución total	
Campo	Descripción
Caso de uso	CU07 – Registrar devolución total
Actores	Usuario autenticado
Tipo	Básico
Propósito	Permitir la devolución completa de un préstamo.
Precondiciones	1. El usuario debe estar autenticado.2. El préstamo no debe estar previamente devuelto.

Postcondiciones	1. El préstamo cambia a estado returned.2. La disponibilidad del libro se actualiza completamente.
Escenario de éxito	1. El usuario accede al préstamo.2. Solicita la devolución total.3. El sistema valida el estado del préstamo.4. El sistema actualiza el estado a returned.5. El sistema restituye la disponibilidad del libro.
Extensiones	El préstamo ya fue devuelto previamente.

Tabla 14. Especificación del caso de uso CU08 – Extender préstamo	
Campo	Descripción
Caso de uso	CU08 – Extender préstamo
Actores	Usuario autenticado
Tipo	Básico
Propósito	Permitir extender la duración de un préstamo activo o parcial.
Precondiciones	1. El usuario debe estar autenticado.2. El préstamo debe encontrarse activo o parcial.3. El préstamo no debe estar devuelto completamente.
Postcondiciones	La fecha de vencimiento del préstamo se incrementa en siete días.
Escenario de éxito	1. El usuario accede al préstamo.2. Solicita la extensión.3. El sistema valida el estado del préstamo.4. El sistema actualiza la fecha de vencimiento.
Extensiones	El préstamo ya fue devuelto y no puede extenderse.

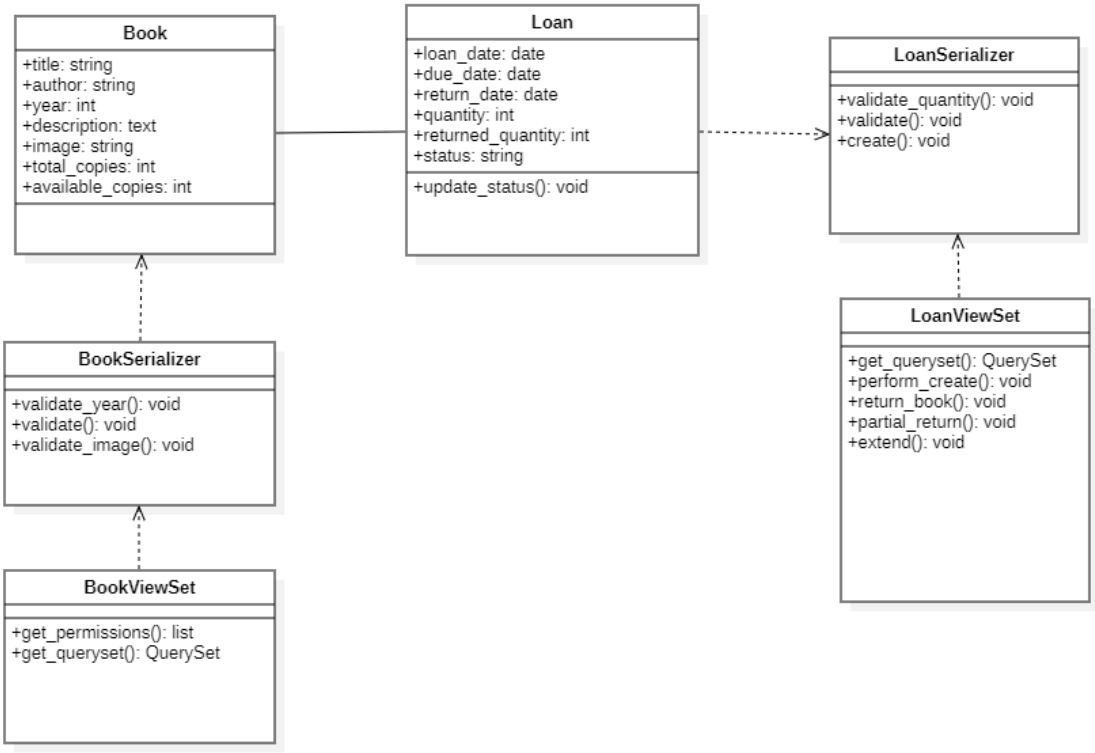
XI. Diagramas de alto nivel

- Diagrama Entidad-Relación



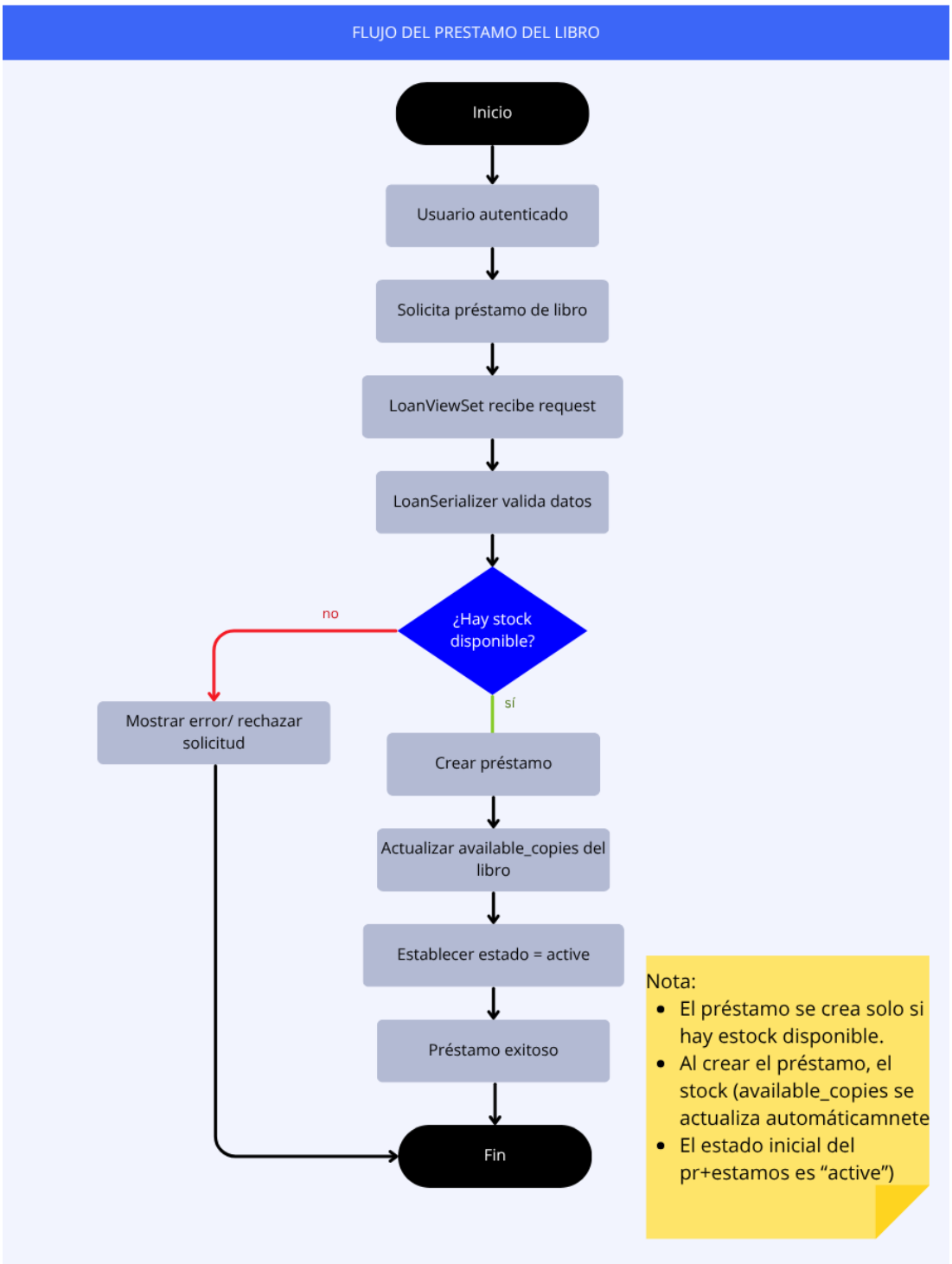
La cardinalidad indica cuántas instancias de una entidad pueden relacionarse con otra. En este caso, un usuario puede tener cero o muchos préstamos, mientras que cada préstamo pertenece a un único usuario.

- Diagrama de Clases



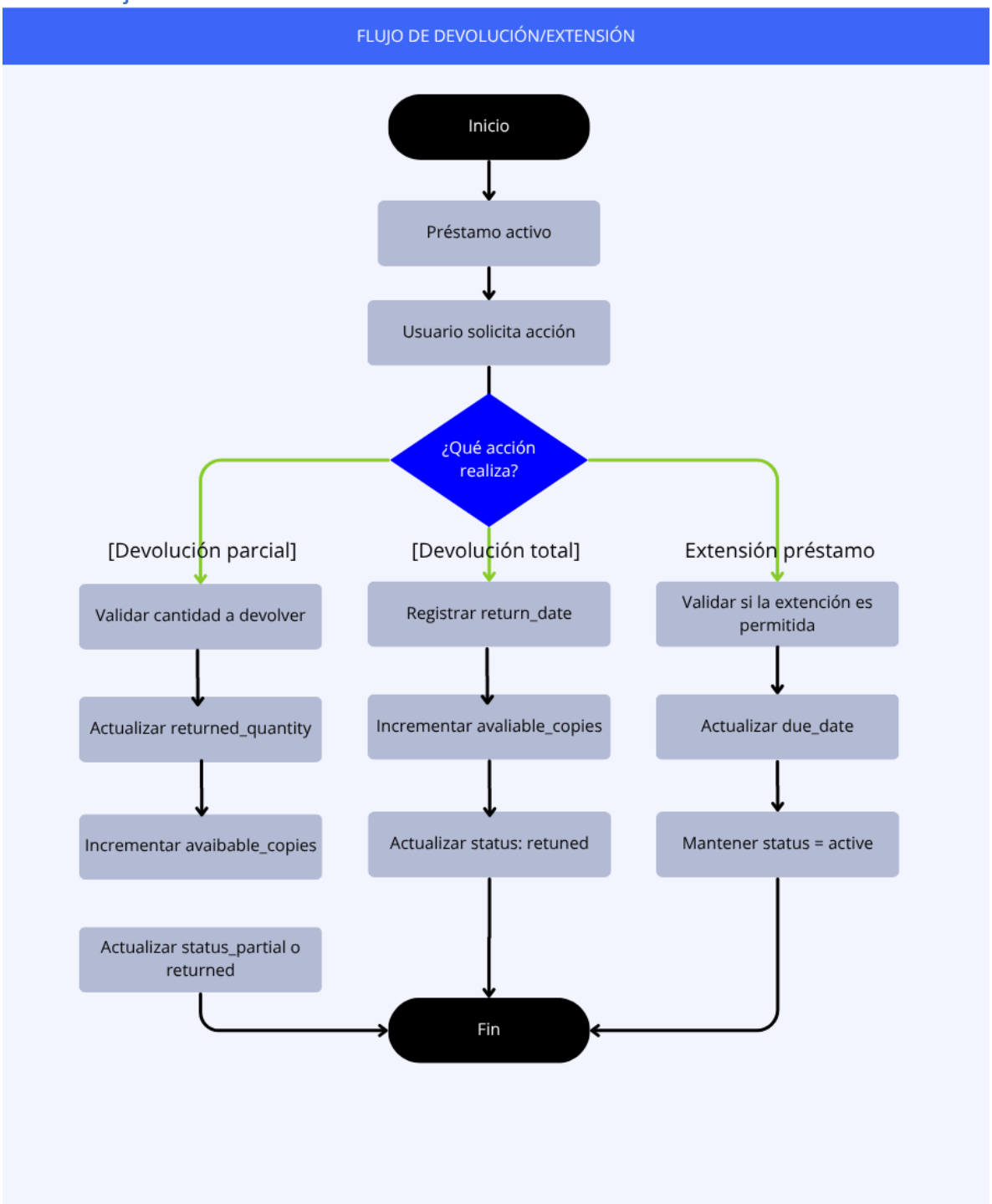
Aquí podemos ver cómo está construido el sistema a nivel de clases. Se identifican los modelos principales como Book y Loan, así como los componentes que manejan la lógica, como los serializers y viewsets, mostrando cómo se conectan para gestionar los préstamos.

- Flujo general del préstamo



Este flujo describe el proceso desde que un usuario solicita un préstamo hasta su registro en el sistema. Incluye la validación de disponibilidad del libro, la creación del préstamo y la actualización del inventario. En caso de no existir stock, el sistema rechaza la solicitud.

- ## Flujo de Devolución



Este flujo contempla las diferentes acciones que un usuario puede realizar sobre un préstamo: devolución parcial, devolución total o extensión.

Cada acción valida el estado del préstamo y actualiza tanto el inventario como el estado correspondiente.

XII. Criterios de aceptación

A continuación, se presentan los criterios de aceptación definidos para las principales funcionalidades del sistema:

- El sistema debe permitir registrar libros únicamente a usuarios con rol administrador.
- El sistema debe permitir consultar el catálogo de libros a cualquier usuario.
- El sistema debe permitir filtrar libros por autor y año.
- El sistema debe permitir registrar préstamos únicamente a usuarios autenticados.
- El sistema debe validar que exista disponibilidad de copias antes de crear un préstamo.
- El sistema debe permitir registrar devoluciones parciales, actualizando correctamente el inventario.
- El sistema debe permitir registrar devoluciones completas, cambiando el estado del préstamo a "returned".
- El sistema debe impedir la extensión de préstamos que ya han sido devueltos.
- El sistema debe permitir extender préstamos activos o parciales.
- El sistema debe restringir el acceso a préstamos únicamente a su propietario o a usuarios administradores.
- El sistema debe validar que las cantidades de préstamo y devolución sean mayores a cero.
- El sistema debe validar que las imágenes cargadas correspondan a archivos válidos y cumplan con las restricciones de tamaño.

XIII. Pruebas (Postman)

Se desarrolló una colección de Postman con el objetivo de validar el correcto funcionamiento de la API mediante diferentes escenarios de prueba.

La colección incluye:

Flujo principal de préstamos, validando la creación, extensión y devolución de libros.

Casos de error, incluyendo validaciones de datos, restricciones de negocio y condiciones inválidas.

Pruebas de seguridad, verificando restricciones de acceso a recursos según el rol del usuario.

Uso de variables de entorno para la gestión dinámica de identificadores y credenciales.

Implementación de scripts de pre-request para la preparación de datos.

Validaciones post-response para verificar códigos de estado y estructura de las respuestas.

Estas pruebas permiten asegurar la consistencia del sistema, la correcta implementación de la lógica de negocio y el cumplimiento de los requerimientos definidos.

XIV. Levantamiento de Proyecto

El sistema puede ejecutarse mediante contenedores Docker, lo que permite su despliegue de forma sencilla y reproducible en diferentes entornos.

A continuación, se describen los pasos necesarios para levantar el proyecto:

1. Clonar el repositorio

```
git clone https://github.com/EliSalas1/library-system.git
```

```
cd library-system
```

2. Configurar variables de entorno

Se incluye un archivo .env.example con la configuración base.

En caso de ser necesario, puede copiarse como .env y ajustarse según el entorno:

```
cp .env.example .env
```

3. Construir y levantar los contenedores

```
docker-compose up --build
```

Este comando realiza lo siguiente:

Construye las imágenes del proyecto

Levanta los servicios definidos (backend y base de datos PostgreSQL)

Inicializa el entorno de ejecución

4. Ejecutar migraciones

```
docker-compose exec web python manage.py migrate
```

Permite crear las tablas necesarias en la base de datos.

5. Crear superusuario

```
docker-compose exec web python manage.py createsuperuser
```

Se utiliza para acceder al panel administrativo de Django.

6. Acceder al sistema

Una vez levantado el proyecto, el sistema estará disponible en:

API: http://localhost:8000/api/

Panel de administración: http://localhost:8000/admin/

Para información adicional y configuración detallada, consultar el archivo README.md incluido en el repositorio.

XV. Conclusión

El sistema desarrollado cumple con los requerimientos funcionales establecidos, proporcionando una solución completa para la gestión de libros y préstamos mediante una API REST.

Se implementaron funcionalidades clave como el control de inventario, la gestión de préstamos, devoluciones parciales y totales, así como la extensión de préstamos, integrando validaciones que garantizan la consistencia de los datos y el cumplimiento de las reglas de negocio.

Adicionalmente, se incorporaron mecanismos de control de acceso basados en roles, asegurando que cada usuario interactúe únicamente con los recursos que le corresponden, fortaleciendo así la seguridad del sistema.

Las pruebas realizadas mediante Postman permitieron validar escenarios funcionales, casos de error y restricciones de seguridad, proporcionando evidencia del correcto comportamiento de la API.

Finalmente, la solución fue desarrollada utilizando Django REST Framework, PostgreSQL y Docker, aplicando buenas prácticas de desarrollo backend y garantizando una estructura escalable, mantenible y fácilmente desplegable.